P0901R3
Size feedback in operator new

Published Proposal, 2019-01-21

This version:
http://wg21.link/P0901R3

Authors:
Andrew Hunter
Chris Kennelly (Google)

Audience:
EWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

Provide access to actual malloc buffer sizes for users.

Table of Contents

§ 1. Motivation

Throughout this document "malloc" refers to the **implementation** *of* `::operator new` *both as fairly standard practice for implementers, and to make clear the distinction between the interface and the implementation.*

Everyone's favorite dynamic data structure, `std::vector`, allocates memory with code that looks something like this (with many details, like `Allocator`, templating for non `char`, and exception safety, elided):

```
void vector::reserve(size_t new_cap) {
  if (capacity_ >= new_cap) return;
  const size_t bytes = new_cap;
  void *newp = ::operator new(new_cap);
  memcpy(newp, ptr_, capacity_);
  ptr_ = newp;
  capacity_ = bytes;
}
```

Consider the sequence of calls:

```
std::vector<char> v;
v.reserve(37);
// ...
v.reserve(38);
```

All reasonable implementations of malloc round sizes, both for alignment requirements and improved performance. It is extremely unlikely that malloc provided us exactly 37 bytes. We do not need to invoke the allocator here...except that we don't know that for sure, and to use the 38th byte would be undefined behavior. We would like that 38th byte to be usable without a roundtrip through the allocator.

This paper proposes an API making it safe to use that byte, and explores many of the design choices (not all of which are obvious without implementation experience.)

§ 1.1. nallocx: not as awesome as it looks

The simplest way to help here is to provide an informative API answering the question "If I ask for N bytes, how many do I actually get?" [jemalloc] calls this `nallocx`. We can then use that hint as a smarter parameter for operator new:

```
void vector::reserve(size_t new_cap) {
  if (capacity_ >= new_cap) return;
  const size_t bytes = nallocx(new_cap, 0);
  void *newp = ::operator new(bytes);
  memcpy(newp, ptr_, capacity_);
  ptr_ = newp;
  capacity_ = bytes;
}
```

This is a good start, and does in fact work to allow vector and friends to use the true extent of returned objects. But there are three significant problems with this approach.

§ 1.1.1. nallocx must give a conservative answer

While many allocators have a deterministic map from requested size to allocated size, it is by no means guaranteed that all do. Presumably they can make a reasonably good guess, but if two calls to `::operator new(37)` might return 64 and 128 bytes, we'd definitely rather know the right answer, not a conservative approximation.

1.1.2. nallocx duplicates work

Allocation is often a crucial limit on performance. Most allocators compute the returned size of an object as part of fulfilling that allocation...but if we make a second call to `nallocx`, we duplicate all that communication, and also the overhead of the function call.

1.1.3. nallocx hides information from malloc

The biggest problem (for the authors) is that `nallocx` discards information malloc finds valuable (the user's intended allocation size.) That is: in our running example, malloc normally knows that the user wants 37 bytes (then 38), but with `nallocx`, we will only ever be told that they want 40 (or 48, or whatever `nallocx(37)` returns.)

Google's malloc implementation (TCMalloc) rounds requests to one of a small (<100) number of *sizeclasses*: we maintain local caches of appropriately sized objects, and cannot do this for every possible size of object. Originally, these sizeclasses were just reasonably evenly spaced among the range they cover. Since then, we have used extensive telemetry on allocator use in the wild to tune these choices. In particular, as we know (approximately) how many objects of any given size are requested, we can solve a fairly simple optimization problem to minimize the total internal fragmentation for any choice of N sizeclasses.

Widespread use of `nallocx` breaks this. By the time TCMalloc's telemetry sees a request that was hinted by nallocx, to the best of our knowledge the user *wants* exactly as many bytes as we currently provide them. If a huge number of callers wanted 40 bytes but were currently getting 48, we'd lose the ability to know that and optimize for it.

Note that we can't take the same telemetry from `nallocx` calls: we have no idea how many times the resulting hint will be used (we might not allocate at all, or we might cache the result and make a million allocations guided by it.) We would also lose important information in the stack traces we collect from allocation sites.

Optimization guided by malloc telemetry has been one of our most effective tools in improving allocator performance. It is important that we fix this issue *without* losing the ground truth of what a caller of `::operator new` wants.

These three issues explain why we don't believe `nallocx` is a sufficient solution here.

§ 1.2. after allocation is too late

Another obvious suggestion is to add a way to inspect the size of an object returned by `::operator new`. Most mallocs provide a way to do this; [jemalloc] calls it `sallocx`. Vector would look like:

```
void vector::reserve(size_t new_cap) {
  if (capacity_ >= new_cap) return;
  void *newp = ::operator new(new_cap);
  const size_t bytes = sallocx(newp);
  memcpy(newp, ptr_, capacity_);
  ptr_ = newp;
  capacity_ = bytes;
}
```

This is worse than nallocx. It fixes the non-constant size problem, and avoids a feedback loop, but the performance issue is worse (this is the major issue *fixed* by [SizedDelete]!), and what's worse, the above code invokes UB as soon as we touch byte `new_cap+1`. We could in principle change the standard, but this would be an implementation nightmare.

§ 1.3. realloc's day has passed

We should also quickly examine why the classic C API `realloc` is insufficient.

```
void vector::reserve(size_t new_cap) {
  if (capacity_ >= new_cap) return;
  ptr_ = realloc(ptr_, new_cap);
  capacity_ = new_cap;
}
```

In principle a realloc from 37 to 38 bytes wouldn't carry the full cost of allocation. But it's dramatically more expensive than making no call at all. What's more, there are a number of more complicated dynamic data structures that store variable-sized chunks of data but are never actually resized. These data structures still deserve the right to use all the memory they're paying for.

Furthermore, `realloc`'s original purpose was not to allow the use of more bytes the caller already had, but to (hopefully) extend an allocation in place to adjacent free space. In a classic malloc implementation this would actually be possible...but most modern allocators use variants of slab allocation. Even if the 65th byte in a 64-byte allocation isn't in use, they cannot be combined into a

single object; it's almost certainly required to be used for the next 64-byte allocation. In the modern world, `realloc` serves little purpose.

§ 2. Proposal

We propose adding new overloads of `::operator new` (size-returning allocation functions) that directly inform the user of the size available to them. C++ makes `::operator new` replaceable (15.5.4.6), allowing a program to provide its own version different from the standard library's implementation.

We propose wording, relative to [N4791]:

- Amend [basic.stc.dynamic.allocation] (6.6.5.4.1) paragraph 1:

> An allocation function shall be a class member function or a global function; a program is ill-formed if an allocation function is declared in a namespace scope other than global scope or declared static in global scope.
>
> An allocation function is a *size-returning allocation function* if it has a second parameter of type `std::return_size_t`, or it has a second parameter of type `std::align_val_t` and a third parameter of type `std::return_size_t`.
>
> The return type shall be `std::sized_ptr_t` if the allocation function is a size-returning allocation function and `void*` otherwise . The first parameter shall have type `std::size_t` ([support.types]). The first parameter shall not have an associated default argument ([dcl.fct.default]). The value of the first parameter is interpreted as the requested size of the allocation. [...]

- Amend [expr.new] (7.6.2.4) paragraph 5:

> Objects created by a *new-expression* have dynamic storage duration ([basic.stc.dynamic]). [Note: The lifetime of such an object is not necessarily restricted to the scope in which it is created. — end note]
>
> When the allocated object is not an array, the result of the new-expression is
>
> - if the new-expression is a size-returning placement new expression (see below), an object of type `std::return_size_t` ([new.syn]) whose `p` member points to the object created and whose `n` member is the `n` member of the value returned by the size-returning allocation function;
> - otherwise, a pointer to the object created.

- Amend [expr.new] (7.6.2.4) paragraph 6:

> When the allocated object is an array (that is, the *noptr-new-declarator* syntax is used or the *new-type-id* or *type-id* denotes an array type), the *new-expression* yields
>
> - if the new-expression is a size-returning placement new expression (see below), an object of type `std::return_size_t` ([new.syn]) whose `p` member points to the object created and whose `n` member is the `n` member of the value returned by the size-returning allocation function less any array allocation overhead;
> - otherwise, a pointer to the initial element (if any) of the array.
>
> [Note: Both `new int` and `new int[10]` have type `int*` and the type of `new int[i][10]` is `int (*)[10]` — end note] The *attribute-specifier-seq* in a *noptr-new-declarator* appertains to the associated array type.

- Amend [expr.new] (7.6.2.4) paragraph 15:

> Overload resolution is performed on a function call created by assembling an argument list. The first argument is the amount of space requested, and has type `std::size_t`. If the type of the allocated object has new-extended alignment, the next argument is the type's alignment, and has type `std::align_val_t`. If the *new-placement* syntax is used, the initializer-clauses in its expression-list are the succeeding arguments. If no matching function is found and the allocated object type has new-extended alignment, the alignment argument is removed from the argument list, and overload resolution is performed again. A *size-returning placement new expression* is one whose selected allocation function is a *size-returning allocation function*.

- Amend [expr.new] (7.6.2.4) paragraph 25:

> If a *new-expression* calls a deallocation function, it passes
>
> - the `p` member of the object returned by the *size-returning allocation function*;
> - otherwise, the value returned from the allocation function call
>
> as the first argument of type `void*`. If a placement deallocation function is called, it is passed the same additional arguments as were passed to the placement allocation function, that is, the same arguments as those specified with the *new-placement* syntax. If no matching function is found and one of the arguments to the *new-placement* syntax has type `std::return_size_t`, that argument is removed from the argument list, and overload resolution is performed again. If the implementation is allowed to introduce a temporary object or make a copy of any argument as part of the call to the allocation function, it is unspecified whether the same object is used in the call to both the allocation and deallocation functions.

- Amend [replacement.functions] (15.5.4.6) paragraph 2:

```
operator new(std::size_t)
operator new(std::size_t, std::align_val_t)
operator new(std::size_t, const std::nothrow_t&)
operator new(std::size_t, std::align_val_t, const std::nothrow_t&)


operator new(size_t size, std::return_size_t)
operator new(size_t size, std::align_val_t al, std::return_size_t)
operator new(size_t size, std::return_size_t, std::nothrow_t)
operator new(size_t size, std::align_val_t al, std::return_size_t, std::not


[...]

operator new[](std::size_t)
operator new[](std::size_t, std::align_val_t)
operator new[](std::size_t, const std::nothrow_t&)
operator new[](std::size_t, std::align_val_t, const std::nothrow_t&)


operator new[](size_t size, std::return_size_t)
operator new[](size_t size, std::align_val_t al, std::return_size_t)
operator new[](size_t size, std::return_size_t, std::nothrow_t)
operator new[](size_t size, std::align_val_t al, std::return_size_t, std::r


[...]
```

- Amend header `<new>` synopsis in [new.syn] (16.6.1):

```
namespace std {
  class bad_alloc;
  class bad_array_new_length;

  struct destroying_delete_t {
    explicit destroying_delete_t() = default;
  };
  inline constexpr destroying_delete_t destroying_delete{};


  struct std::return_size_t {};

  template<typename T = void>
  struct std::sized_ptr_t {
    T *p;
    size_t n;
  };


  enum class align_val_t : size_t {};

  [...]
}

[[nodiscard]] void* operator new(std::size_t size);
[[nodiscard]] void* operator new(std::size_t size, std::align_val_t alignme
[[nodiscard]] void* operator new(std::size_t size, const std::nothrow_t&) r
[[nodiscard]] void* operator new(std::size_t size, std::align_val_t alignme
                                 const std::nothrow_t&) noexcept;


[[nodiscard]] std::sized_ptr_t ::operator new(size_t size, std::return_size
[[nodiscard]] std::sized_ptr_t ::operator new(
  size_t size, std::align_val_t alignment, std::return_size_t);
[[nodiscard]] std::sized_ptr_t ::operator new(
  size_t size, std::return_size_t, std::nothrow_t) noexcept;
[[nodiscard]] std::sized_ptr_t ::operator new(
  size_t size, std::align_val_t alignment, std::return_size_t, std::nothrow

[...]

[[nodiscard]] void* operator new[](std::size_t size);
[[nodiscard]] void* operator new[](std::size_t size, std::align_val_t align
```

```
[[nodiscard]] void* operator new[](std::size_t size, const std::nothrow_t&)
[[nodiscard]] void* operator new[](std::size_t size, std::align_val_t alignr
                                   const std::nothrow_t&) noexcept;


[[nodiscard]] std::sized_ptr_t ::operator new[](size_t size, std::return_si
[[nodiscard]] std::sized_ptr_t ::operator new[](
  size_t size, std::align_val_t alignment, std::return_size_t);
[[nodiscard]] std::sized_ptr_t ::operator new[](
  size_t size, std::return_size_t, std::nothrow_t) noexcept;
[[nodiscard]] std::sized_ptr_t ::operator new[](
  size_t size, std::align_val_t alignment, std::return_size_t, std::nothrow

[...]

[[nodiscard]] void* operator new  (std::size_t size, void* ptr) noexcept;
[[nodiscard]] void* operator new[](std::size_t size, void* ptr) noexcept;
void  operator delete  (void* ptr, void*) noexcept;
void  operator delete[](void* ptr, void*) noexcept;
```

- Amend [new.delete.single] (16.6.2.1)

```
[[nodiscard] std::sized_ptr_t ::operator new(size_t size, std::return_s
[[nodiscard] std::sized_ptr_t ::operator new(
  size_t size, std::align_val_t alignment, std::return_size_t);
```

- *Effects*: Same as above, except these are called by a placement version of a *new-expression* when a C++ program prefers the *size-returning allocation function*.
- *Replaceable*: A C++ program may define functions with either of these function signatures, and thereby displace the default versions defined by the C++ standard library.
- *Required behavior*: Return a sized_ptr_t whose `p` member represents the address of a region of *N* bytes of suitably aligned storage ([basic.stc.dynamic]) for some *N* >= `size`, and whose `n` member is *N*, or else throw a `bad_alloc` exception. This requirement is binding on any replacement versions of these functions.
- *Default behavior*: Returns `std::sized_ptr_t{operator new(size), N}` and `std::sized_ptr_t{operator new(size, alignment), N}` respectively. If a user-provided operator new is invoked directly or indirectly, N is `size`.

```
[[nodiscard] std::sized_ptr_t ::operator new(
  size_t size, std::return_size_t, std::nothrow_t) noexcept;
[[nodiscard] std::sized_ptr_t ::operator new(
  size_t size, std::align_val_t alignment, std::return_size_t, std::noth
```

- *Effects*: Same as above, except these are called by a placement version of a *new-expression* when a C++ program prefers the *size-returning allocation function* and a null pointer result as an error indication.
- *Replaceable*: A C++ program may define functions with either of these function signatures, and thereby displace the default versions defined by the C++ standard library.
- *Required behavior*: Return a sized_ptr_t whose `p` member represents the address of a region of *N* bytes of suitably aligned storage ([basic.stc.dynamic]) for some *N* >= `size`, and whose `n` member is *N*, or else return `std::sized_ptr_t{nullptr, 0}`. Each of these nothrow versions of `operator new` returns a pointer obtained as if acquired from the (possibly replaced) corresponding non-placement function. This requirement is binding on any replacement versions of these functions.
- *Default behavior*: Returns `std::sized_ptr_t{operator new(size), N}` and `std::sized_ptr_t{operator new(size, alignment), N}` respectively. If a user-

provided operator new is invoked directly or indirectly, N is `size`. If the call to `operator new` throws, returns `std::sized_ptr_t{nullptr, 0}`.

```
void operator delete(void* ptr) noexcept;
void operator delete(void* ptr, std::size_t size) noexcept;
void operator delete(void* ptr, std::align_val_t alignment) noexcept;
void operator delete(void* ptr, std::size_t size, std::align_val_t align
```

[...]

- *Requires*: If the `alignment` parameter is not present, `ptr` shall have been returned by an allocation function without an alignment parameter. If present, the `alignment` argument shall equal the alignment argument passed to the allocation function that returned `ptr`. If present, the `size` argument shall equal the `size` argument passed to the allocation function that returned `ptr` , if `ptr` was not allocated by a *size-returning allocation function*. If present, the `size` argument shall satisify `n >= size >= requested` if `ptr` was allocated by a *size-returning allocation function*, where `n` is the size returned in `std::sized_ptr_t` and `requested` is the size argument passed to the allocation function .

[...]

- Amend [new.delete.array] (16.6.2.2)

```
[[nodiscard] std::sized_ptr_t ::operator new[](size_t size, std::return
[[nodiscard] std::sized_ptr_t ::operator new[](
  size_t size, std::align_val_t alignment, std::return_size_t);
```

- *Effects*: Same as above, except these are called by a placement version of a *new-expression* when a C++ program prefers the *size-returning allocation function*.
- *Replaceable*: A C++ program may define functions with either of these function signatures, and thereby displace the default versions defined by the C++ standard library.
- *Required behavior*: Return a sized_ptr_t whose `p` member represents the address of a region of *N* bytes of suitably aligned storage ([basic.stc.dynamic]) for some *N* >= `size`, and whose `n` member is *N*, or else throw a `bad_alloc` exception. This requirement is binding on any replacement versions of these functions.
- *Default behavior*: Returns `std::sized_ptr_t{operator new[](size), N}` and `std::sized_ptr_t{operator new[](size, alignment), N}` respectively. If a user-provided operator new is invoked directly or indirectly, N is `size`.

```
[[nodiscard] std::sized_ptr_t ::operator new[](
  size_t size, std::return_size_t, std::nothrow_t) noexcept;
[[nodiscard] std::sized_ptr_t ::operator new[](
  size_t size, std::align_val_t alignment, std::return_size_t, std::noth
```

- *Effects*: Same as above, except these are called by a placement version of a *new-expression* when a C++ program prefers the *size-returning allocation function* and a null pointer result as an error indication.
- *Replaceable*: A C++ program may define functions with either of these function signatures, and thereby displace the default versions defined by the C++ standard library.
- *Required behavior*: Return a sized_ptr_t whose `p` member represents the address of a region of *N* bytes of suitably aligned storage ([basic.stc.dynamic]) for some *N* >= `size`, and whose `n` member is *N*, or else return `std::sized_ptr_t{nullptr, 0}`. This requirement is binding on any replacement versions of these functions.
- *Default behavior*: Returns `std::sized_ptr_t{operator new[](size), N}` and `std::sized_ptr_t{operator new[](size, alignment), N}` respectively. If a user-provided operator new is invoked directly or indirectly, N is `size`. If the call to `operator new[]` throws, returns `std::sized_ptr_t{nullptr, 0}`.

```
void operator delete[](void* ptr) noexcept;
void operator delete[](void* ptr, std::size_t size) noexcept;
void operator delete[](void* ptr, std::align_val_t alignment) noexcept;
void operator delete[](void* ptr, std::size_t size, std::align_val_t ali
```

[...]

- *Requires*: If the `alignment` parameter is not present, `ptr` shall have been returned by an allocation function without an `alignment` parameter. If present, the `alignment` argument shall equal the `alignment` argument passed to the allocation function that returned `ptr`. If present, the `size` argument shall equal the `size` argument passed to the allocation function that returned `ptr` , if `ptr` was not allocated by a *size-returning allocation function*. If present, the `size` argument shall satisify `n >= size >= requested` if `ptr` was allocated by a *size-returning allocation function*, where `n` is the size returned in `std::sized_ptr_t` and `requested` is the size argument passed to the allocation function .

[...]

§ 3. Alternative Designs Considered

Another signature we could use would be:

```
enum class return_size_t : std::size_t {};
void* ::operator new(size_t size, std::return_size_t);
```

(and so on.) This is slightly simpler to read as a signature, but arguably worse in usage:

```
std::tie(obj.ptr, obj.size) = ::operator new(37, std::return_size_t{});

// ...vs...

// Presumably the object implementation wants to contain a size_t,
// not a return_size_t.
std::return_size_t rs;
obj.ptr = ::operator new(37, rs);
obj.size = rs;
```

More importantly, this form is less efficient. In practice, underlying malloc implementations provide actual definitions of `::operator new` symbols which are called like any other function. Passing a reference parameter requires us to actually return the size via memory.

- Linux ABIs support returning at least two scalar values in registers (even if they're members of a trivially copyable struct) which can be dramatically more efficient.
- The [MicrosoftABI] returns large types by pointer, but this is no worse than making the reference parameter an inherent part of the API.

Whether we use a reference parameter or a second returned value, the interpretation is the same.

§ 3.1. *How* many `::operator new`'s?

It is unfortunate that we have so many permutations of `::operator new`--eight seems like far more than we should really need! But there really isn't any significant runtime cost for having them. Use of raw calls to `::operator new` is relatively rare: It's a *building block* for low-level libraries, allocators ([P0401]), and so on, so the cognitive burden on C++ users is low.

The authors have considered other alternatives to the additional overloads. At the Jacksonville meeting, EWG suggested looking at parameter packs.

- Parameter packs do not reduce the number of symbols introduced. Implementers still need to provide implementations each of the n overloads.
- Retrofitting parameter packs leaves us with *more* mangled variants. Implementers need to provide both the legacy symbols as well as the parameter pack-mangled symbols.

The authors have also considered APIs where all parameters are passed, thereby requiring a single new overload. This adds further overhead for implementations, as it moves compile-time decisions (is the alignment at or below the minimum guaranteed by `operator new`) into runtime ones.

The alternative to modifying the handling of *new-expressions* invoking deallocation functions (when an exception is thrown) would require additional overloads for `operator delete` / `operator delete[]` whose sole purpose would be to accept and discard the `std::return_size_t`.

§ 3.2. Implementation difficulty

It's worth reiterating that there's a perfectly good trivial implementation of these functions:

```
std::sized_ptr_t ::operator new(size_t n, std::return_size_t) {
  return {::operator new(n), n};
}
```

Malloc implementations are free to properly override this with a more impactful definition, but this paper poses no significant difficulty for toolchain implementers.

Implementation Experience:

- TCMalloc has developed a (currently internal) implementation. While this requires mapping from an integer size class to the true number of bytes, combining this lookup with the allocation is more efficient as we avoid recomputing the sizeclass itself (given a request) or deriving it from the object's address.
- jemalloc is prototyping a `smallocx` function providing a C API for this functionality [smallocx].

§ 3.3. Interaction with Sized Delete

For allocations made with `sized_ptr_t`-returning `::operator new`, we need to relax `::operator delete`'s size argument (16.6.2.1 and 16.6.2.2). For allocations of `T`, the size quanta used by the allocator may not be a multiple of `sizeof(T)`, leading to both the original and returned sizes being unrecoverable at the time of deletion.

Consider the memory allocated by:

```
using T = std::aligned_storage<16, 8>::type;

std::vector<T> v(4);
```

The underlying heap allocation is made with `::operator new(64, std::return_size_t)`.

- The memory allocator may return a 72 byte object: Since there is no `k` such that `sizeof(T) * k = 72`, we can't provide that value to `::operator delete(void*, size_t)`. The only option would be storing 72 explicitly, which would be wasteful.
- The memory allocator may instead return an 80 byte object (5 `T`'s): We now cannot represent the original request when deallocating without additional storage.

For allocations made with

```
std::tie(p, m) = ::operator new(n, std::return_size_t{});
```

we permit `::operator delete(p, s)` where `n <= s <= m`.

This behavior is consistent with [jemalloc]'s `sdallocx`, where the deallocation size must fall between the request (`n`) and the actual allocated size (`m`) inclusive.

§ 3.4. Advantages

It's easy to see that this approach nicely solves the problems posed by other methods:

- We pay almost nothing in speed to return an actual-size parameter. For TCMalloc and jemalloc, this is typically a load from to map from *sizeclass* to size. This cost is strictly smaller than with `nallocx` or the like, as that same translation must be performed in addition to the duplicative work previously discussed.
- We are told exactly the size we have, without risk of UB. We can avoid subsequent reallocations when growing to a buffer to an already-allocated size.
- Allocator telemetry knows actual request sizes exactly.

§ 4. New Expressions

Additionally, we propose expanding this functionality to `new` expressions by returning:

- For `new`, a pointer to the object created and the size of the allocation in bytes.
- For `new[]`, a pointers to the initial element of the array and the size of the allocation in bytes (less overhead):

```
auto [p, sz] = new (std::return_size_t) T[5];
for (int i = 5; i < sz / sizeof(T); i++) {
  new (p[i]) T;
}
for (int i = 0; i < sz / sizeof(T); i++) {
  p[i].DoStuff();
}
for (int i = 5; i < sz / sizeof(T); i++) {
  p[i].~T();
}
delete[] p;
```

We considered alternatives for returning the size.

- We could return two pointers, the initial object and one past the end of the array (minus the array allocation overhead).

  ```
  auto [start, end] = new (std::return_size_t) T[5];
  for (T* p = start + 5; p != end; p++) {
      new (p) T;
  }
  for (T* p = start; p != end; p++) {
      p->DoStuff();
  }
  for (T* p = start + 5; p != end; p++) {
      p->~T();
  }
  delete[] start;
  ```

  The pair of pointers provides convience for use with iterator-oriented algorithms. The problem we foresee is that a size-returning allocation function may not provide a size that is an appropriate multiple of `sizeof(T)` (or fail to be a multiple of `alignof(T)`, a possible, but unlikely scenario). This imposes a need for more extensive logic around the *new-expression* in handling the returned allocation size.

- We could return the size in units of `T`, this leads to an inconsistency between the expected usage for `new` and `new[]`:

  - For `new`, we may only end up fitting a single `T` into an allocator size quanta, so the extra space remains unusable. If we can fit multiple `T` into a single allocator size quanta, we now

have an array from what was a scalar allocation site. This cannot be foreseen by the compiler as `::operator new` is a replaceable function.

  - For `new[]`, the size in units of `T` can easily be derived from the returned size in bytes.

- We could pass the size in units of `T` or bytes to the constructor of `T`:

  - For `new`, this is especially useful for tail-padded arrays, but neglects default-initialized `T`.
  - For `new[]`, a common use case is expected to be the allocation of arrays of `char`, `int`, etc. The size of the overall array is irrelevant for the individual elements.

- We could return the size via a reference parameter:

```
std::return_end<T> end;
T* p = new (end) T[5];
for (T* p = start + 5; p != end; p++) {
    new (p) T;
}
for (T* p = start; p != end; p++) {
    p->DoStuff();
}
for (T* p = start + 5; p != end; p++) {
    p->~T();
}
```

or, demonstrated with bytes:

```
std::return_size_t size;
T* p = new (s) T[5];
for (int i = 5; i < size / sizeof(T); i++) {
  new (p[i]) T;
}
for (int i = 0; i < size / sizeof(T); i++) {
  p[i].DoStuff();
}
for (int i = 5; i < size / sizeof(T); i++) {
  p[i].~T();
}
delete[] p;
```

(Casts omitted for clarity.)

As discussed for `::operator new` in §2 Proposal, a reference parameter poses difficulties for optimizers and involves returning the size via memory (depending on ABI).

For `new[]` expressions, we considered alternatively initializing the returned (`sz / sizeof(T)`) number of elements.

- This would avoid the need to explicitly construct / destruct the elements with the additional returned space (if any).

  The *new-initializer* is invoked for the returned number of elements, rather than the requested number of elements. This allows `delete[]` to destroy the correct number of elements (by storing `sz / sizeof(T)` in the array allocation overhead).
- The presented proposal (leaving this space uninitialized) was chosen for consistency with `new`.

§ 5. Related work

[AllocatorExt] considered this problem at the level of the `Allocator` concept. Ironically, the lack of the above API was one significant problem: how could an implementation of `std::allocator` provide the requested feedback in a way that would work with any underlying malloc implementation?

If this proposal is accepted, it's likely that [AllocatorExt] should be taken up again.

§ 6. History

§ 6.1. R2 → R3

- Added proposed wording.
- For newly added allocation functions, `std::nothrow_t` is passed by value, rather than reference.

§ 6.2. R1 → R2

Applied feedback from San Diego Mailing

- Moved from passing `std::return_size_t` parameter by reference to by value. For many ABIs, this is more optimizable and to the authors' knowledge, no worse on any other.
- Added rationale for not using parameter packs for this functionality.

§ 6.3. R0 → R1

Applied feedback from [JacksonvilleMinutes].

- Clarified in §2 Proposal the desire to leverage the existing "replacement functions" wording of the IS, particularly given the close interoperation with the existing `::operator new`/`::operator delete` implementations.
- Added a discussion of the Microsoft ABI in §2 Proposal.
- Noted in §3.1 How many ::operator new's? the possibility of using a parameter pack.
- Added a proposal for §4 New Expressions, as requested by EWG.

Additionally, a discussion of §3.3 Interaction with Sized Delete has been added.

§ References

§ Informative References

[AllocatorExt]
Jonathan Wakely. Extensions to the Allocator interface. 2015-07-08. URL: http://wg21.link/P0401R0

[JacksonvilleMinutes]
Jacksonville 2018 minutes. 2018-03-15. URL: http://wiki.edg.com/bin/view/Wg21jacksonville2018/P0901R0-Jax18

[JEMALLOC]
jemalloc(3) - Linux man page. URL: http://jemalloc.net/jemalloc.3.html

[MicrosoftABI]
Return Values. 2016-11-03. URL: https://docs.microsoft.com/en-us/cpp/build/return-values-cpp

[N4791]
Working Draft, Standard for Programming Language C++. 2018-12-07. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/n4791.pdf

[P0401]
Extensions to the Allocator interface. 2015-07-08. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0401r0.html

[SizedDelete]
L; et al. C++ Sized Deallocation. URL: http://wg21.link/n3536

[SMALLOCX]

Add experimental API to support P0901r0. URL: https://github.com/jemalloc/jemalloc/pull/1270